

তুমি যদি **LangChain** ভালোভাবে শিখতে চাও, তাহলে **Structured Output** হচ্ছে এমন একটি কনসেপ্ট যেটা না বুঝলে AI Application, AI Agent, RAG, Tool Calling, API Development—এসব ঠিকভাবে করা যাবে না।

আমি একদম শূন্য (**Beginner**) থেকে **Advanced** পর্যন্ত সহজ ভাষায় বুঝিয়ে দিচ্ছি।

প্রথমে একটা বাস্তব উদাহরণ

ধরো তুমি একজন শিক্ষক।

তুমি একজন ছাত্রকে জিজ্ঞেস করলে,

তোমার নাম কী?

সে উত্তর দিল

আমার নাম সাখাওয়াত হোসেন। আমি Comilla University-তে ICT-তে পড়ি। আমার CGPA 3.85।

এটা মানুষের জন্য বোঝা খুব সহজ।

কিন্তু কম্পিউটারের জন্য?

কম্পিউটার তো জানে না

কোনটা নাম

কোনটা ইউনিভার্সিটি

কোনটা CGPA

কম্পিউটার শুধু একটা বড় Text দেখছে।

এটাকে বলে

Unstructured Output

"My name is Sakhawat Hossen. I study ICT at Comilla University. My CGPA is 3.85."

এখানে

সবকিছু একসাথে মিশে আছে।

কিন্তু যদি **AI** এমন উত্তর দেয়

```
{  
  "name":"Sakhawat Hossen",  
  "university":"Comilla University",  
  "department":"ICT",  
  "cgpa":3.85  
}
```

এখন?

কম্পিউটার খুব সহজে বুঝতে পারবে

name কোথায়

cgpa কোথায়

department কোথায়

এটাই হলো

Structured Output

Structured মানে কী?

Structured মানে

আগে থেকেই একটা Structure ঠিক করে দেওয়া।

যেমন

আমরা স্কুলে ভর্তি ফর্ম পূরণ করি।

Name:

Age:

Class:

Roll:

এখানে

যেখানে Name লিখতে হবে

সেখানে Age লিখতে পারবে না।

এটা Structured।

আর যদি শুধু বলি

নিজের সম্পর্কে লিখ।

তাহলে কেউ

৫ লাইন লিখবে

কেউ ১০ লাইন

কেউ নামই লিখবে না।

এটা হলো

Unstructured।

কেন **Structured Output** দরকার?

ধরো তুমি একটি AI Resume Parser বানাচ্ছ।

User Resume Upload করল।

Resume

Sakhawat Hossen

Phone: 017xxxxxxx

Email:

abc@gmail.com

Skills

Python

Machine Learning

Deep Learning

AI যদি Output দেয়

He knows Python and Machine Learning.

তাহলে Database-এ Insert করবে কীভাবে?

Phone কোথায়?

Email কোথায়?

Skills কোথায়?

বোঝা যাচ্ছে না।

কিন্তু যদি Output হয়

```
{  
  "name": "Sakhawat Hossen",  
  "phone": "017xxxx",  
  "email": "abc@gmail.com",  
  "skills": [  
    "Python",  
    "Machine Learning",  
    "Deep Learning"  
  ]  
}
```

এখন Database-এ Insert করা খুব সহজ।

আরেকটি **Example**

ধরো

Amazon Review

This laptop is amazing.

Battery is excellent.

Display is beautiful.

But speaker quality is poor.

তুমি AI-কে বললে

Review Analyze করো।

Unstructured Output

Overall this laptop is very good.

এতে Program কিছুই করতে পারবে না।

Structured Output

```
{
  "topic": "Laptop",
  "pros": [
    "Battery",
    "Display"
  ],
  "cons": [
    "Speaker"
  ],
  "sentiment": "Positive"
}
```

এখন

Website

Dashboard

Analytics

সব জায়গায় Use করা যাবে।

আরেকটা **Example**

ধরো তুমি Food Delivery App বানাচ্ছ।

User লিখল

I want two burgers and one coke.

AI যদি বলে

Sure.

I'll order that.

Program বুঝবে?

না।

Structured Output

```
{  
  "burger":2,  
  "coke":1  
}
```

এখন API-তে পাঠানো যাবে।

AI কেন Default-এ Structured Output দেয় না?

কারণ

LLM-এর কাজ

Language Generate করা।

ওর Nature হলো

Paragraph লেখা।

Example

Explain Machine Learning

LLM

Machine Learning is a branch of AI...

Paragraph লিখবে।

কারণ ও Language Model।

Machine Model না।

তাহলে **LangChain** কী করল?

LangChain বলল

তুমি Paragraph লিখবে না।

এই Format-এ লিখবে।

এটাই

Structured Output।

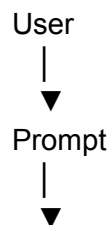
LangChain কীভাবে এটা করে?

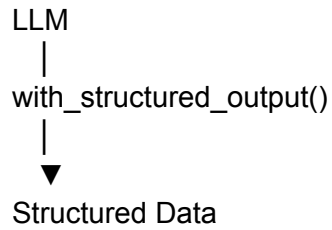
LangChain-এর Function

`with_structured_output()`

এটা Model-এর উপর Wrap হয়ে বসে।

Diagram:





Example

Normal

AI

Rahim
Age 20
Student

Structured

```
{  
  "name": "Rahim",  
  "age": 20,  
  "occupation": "Student"  
}
```

কিন্তু **LangChain** জানবে কীভাবে কোন **Field** লাগবে?

তোমাকেই বলতে হবে।

যেমন

Name

Age

Department

এই Structure আগে Define করতে হবে।

এই Define করার তিনটি উপায় আছে।

Method 1

TypedDict

এটা সবচেয়ে Simple।

```
class Student
```

```
    name
```

```
    age
```

```
    cgpa
```

মানে

AI-কে বলা হচ্ছে

এই তিনটা Field Return করবে।

ধরো Output

```
{  
"name": "Rahim",  
"age": 20,  
"cgpa": 3.7  
}
```

Done.

কিন্তু সমস্যা আছে।

যদি AI লিখে

```
{  
  "name": "Rahim",  
  "age": "Twenty"  
}
```

TypedDict কিছু বলবে না।

যদি লিখে

cgpa=100

তাও Error দিবে না।

এটাতে Validation নেই।

Validation কী?

Validation মানে

ডাটা ঠিক আছে কিনা Check করা।

Example

Age

20

ঠিক।

Age

Apple

ভুল।

Validation বলবে

Error

এজন্য Pydantic

এটাই সবচেয়ে Powerful।

ধরো

তুমি Student বানাও।

Name

String

Age

Integer

CGPA

Float

AI Output দিল

Age

Twenty

Pydantic বলবে

না

Age Integer হওয়া উচিত।

Error দিবে।

আরেকটি **Example**

তুমি বললে

CGPA

0-4

AI দিল

5.5

Pydantic

Invalid

মানে

ডাটা Valid না হলে

Program Stop করে দিবে।

Type Coercion কী?

এটা খুব Interesting।

ধরো

AI লিখল

"20"

String

কিন্তু

তোমার দরকার

Integer

Pydantic নিজেই

"20"

↓

20

করে দিবে।

এটাকে বলে

Type Coercion।

আরেকটা

"3.75"

↓

3.75

Float

নিজেই Convert।

Field()

এটা Description দেওয়ার জন্য।

Example

Age

Student age

বা

CGPA

Must be between 0 and 4

এতে Model আরও ভালোভাবে বুঝতে পারে কী ধরনের মান দিতে হবে, আর Validation-এর নিয়মও নির্ধারণ করা যায়।

JSON Schema কী?

ধরো

তোমার Backend

Python

Frontend

JavaScript

Database

MongoDB

সবাই একই Structure Follow করবে।

তখন

TypedDict

চলবে না।

Pydantic

Python Specific।

তখন Use হয়

JSON Schema

যেটা Universal।

Example

```
{  
  type
```

```
object
```

```
properties
```

```
  name
```

```
    string
```

```
  age
```

```
  integer  
}
```

এটা Python

Java

JavaScript

Go

সব Language বুঝতে পারে।

Output পাওয়ার দুইটি Method

১. JSON Mode

AI শুধু JSON Return করবে।

```
{  
  "name": "Rahim"  
}
```

এটাই।

২. Function Calling

এটা AI Agent-এর জন্য।

ধরো

তোমার Calculator আছে।

Function

add()

AI লিখবে না

2+5=7

বরং

add

2

5

অর্থাৎ Function Call করার মতো Structured তথ্য দেবে, যাতে Agent সরাসরি সেই Function চালাতে পারে।

কেন **Function Calling** দরকার?

AI Agent Flow:

User



Calculate 25×15



LLM



Tool Calling



Calculator



75



LLM



Answer

এখানে যদি AI শুধু Text লিখত,

Calculator বুঝত না।

তাই Function Calling।

Output Parser কী?

সব Model Structured Output Support করে না।

Example

কিছু ছোট Open Source Model।

তখন

AI Output দেয়

Name Rahim

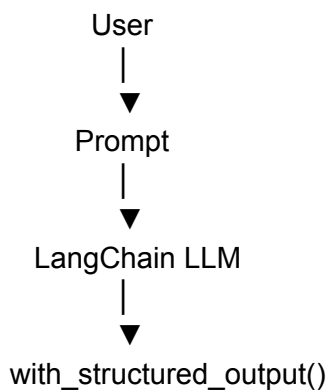
Age 20

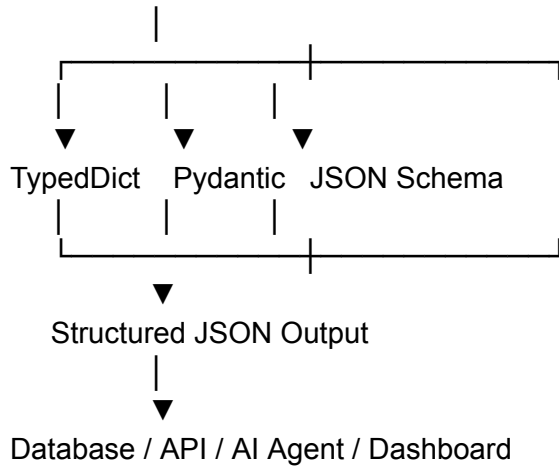
Output Parser এটাকে Convert করে

```
{  
  "name": "Rahim",  
  "age": 20  
}
```

এটাই Output Parser-এর কাজ।

পুরো **Flow** (মনে রাখার জন্য)





কখন কোনটি ব্যবহার করবে?

পরিস্থিতি	কী ব্যবহার করবে?	কেন
ছোট বা দ্রুত Prototype	TypedDict	সহজ, কম কোড
Production Application	Pydantic	Validation, Type Conversion, Constraint—সবই আছে
Python + JavaScript বা Multi-language Project	JSON Schema	সব Language-এ একই Schema ব্যবহার করা যায়
শুধু JSON আউটপুট দরকার	JSON Mode	সরাসরি JSON পাওয়া যায়
AI Agent বা Tool ব্যবহার করবে	Function Calling	Tool/Function-এ Structured Data পাঠানোর জন্য সবচেয়ে উপযোগী

সংক্ষেপে মূল ধারণা

একটি LLM স্বাভাবিকভাবে মানুষের জন্য লেখা **Text** তৈরি করে, কিন্তু বাস্তব অ্যাপ্লিকেশন, Database, API, বা AI Agent-এর জন্য প্রয়োজন হয় **Machine-readable structured data**। LangChain-এর `with_structured_output()` এই সমস্যার সমাধান করে। তুমি আগে থেকে Output-এর Schema (TypedDict, Pydantic, বা JSON Schema) নির্ধারণ করে দাও, আর LLM সেই Structure অনুযায়ী উত্তর দেওয়ার চেষ্টা করে। এর ফলে AI-এর Output সরাসরি কোড, API, Database বা অন্য সফটওয়্যারে নির্ভরযোগ্যভাবে ব্যবহার করা যায়।